Relatório/Manual de limpeza da base RAIS

1. Objetivo do processo

O objetivo dessa etapa foi gerar uma base limpa contendo somente os dados de São José dos Campos. A base original da RAIS possui informações de muitos municípios e é muito grande, então primeiro foi feito o recorte do município e depois a retirada das linhas 100% duplicadas.

Nessa etapa não foi feita a identificação das descrições dos códigos CNAE, natureza jurídica, subsetor do IBGE e outras classificações. Essa parte ficou para depois da limpeza, quando o restante do grupo iria relacionar os códigos com suas respectivas descrições para utilizar nas análises e no Power BI.

2. Arquivos que utilizei

- RAIS_ESTAB_PUB.txt: base original utilizada no processo.
- limpar_rais.py: arquivo Python onde fica o código de limpeza.
- Python instalado no computador.
- Biblioteca Pandas instalada.

Durante o processo foram criados dois novos arquivos de dados:

- RAIS_SJC.csv: primeira base gerada, já contendo somente São José dos Campos.
- RAIS_SJC_LIMPA.csv: base final, somente de São José dos Campos e sem as linhas 100% duplicadas.

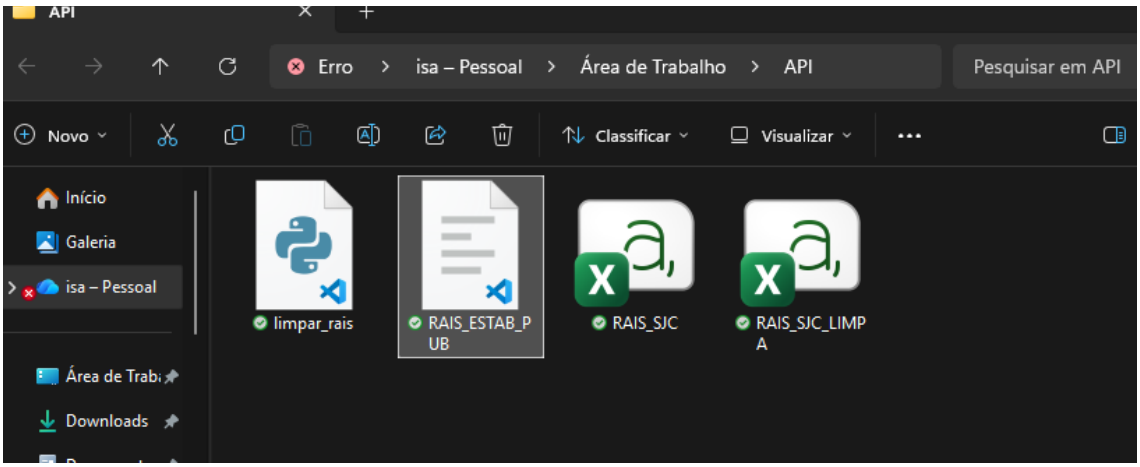


Figura 1 - Pasta do projeto contendo o código, a base original e os dois arquivos gerados.

3. Organização da pasta

Antes de executar o código, os arquivos precisam estar na mesma pasta. No meu caso a pasta utilizada foi a pasta API, localizada na Área de Trabalho. Dentro dela ficaram a base original e o arquivo limpar_rais.py. Depois da execução, os dois arquivos CSV também aparecem nessa mesma pasta.

Importante: se o terminal estiver em outra pasta, o Python não vai encontrar o arquivo limpar_rais.py. Mais abaixo tem a parte mostrando como entrar na pasta certa.

4. Começo do código: importação e seleção das colunas

```
import pandas as pd

arquivo = "RAIS_ESTAB_PUB.txt"
```

A primeira linha importa a biblioteca Pandas e dá a ela o nome pd. O Pandas é usado para trabalhar com tabelas e bases de dados. Nesse processo ele foi usado para ler a RAIS, selecionar colunas, filtrar o município, procurar duplicados e criar os novos arquivos.

Depois é criada a variável arquivo, que guarda o nome da base original. Isso evita ter que escrever o nome completo do arquivo várias vezes no código.

```
colunas = [
    "Município",
    "CNAE 2.0 Classe",
    "CNAE 2.0 Subclasse",
    "Qtd Vínculos CLT",
    "Qtd Vínculos Ativos",
    "Qtd Vínculos Estatutários",
    "Ind Atividade Ano",
    "Ind Estab Participa PAT",
    "Ind Rais Negativa",
    "Ind Simples",
    "Natureza Jurídica",
    "Tamanho Estabelecimento",
    "Tipo Estab",
    "UF",
    "IBGE Subsetor",
    "CEP Estab"
]
```

A lista colunas contém somente os campos que seriam utilizados no projeto. Como a base original possui várias informações que não eram necessárias para o recorte de São José dos Campos, o código já lê apenas essas colunas.

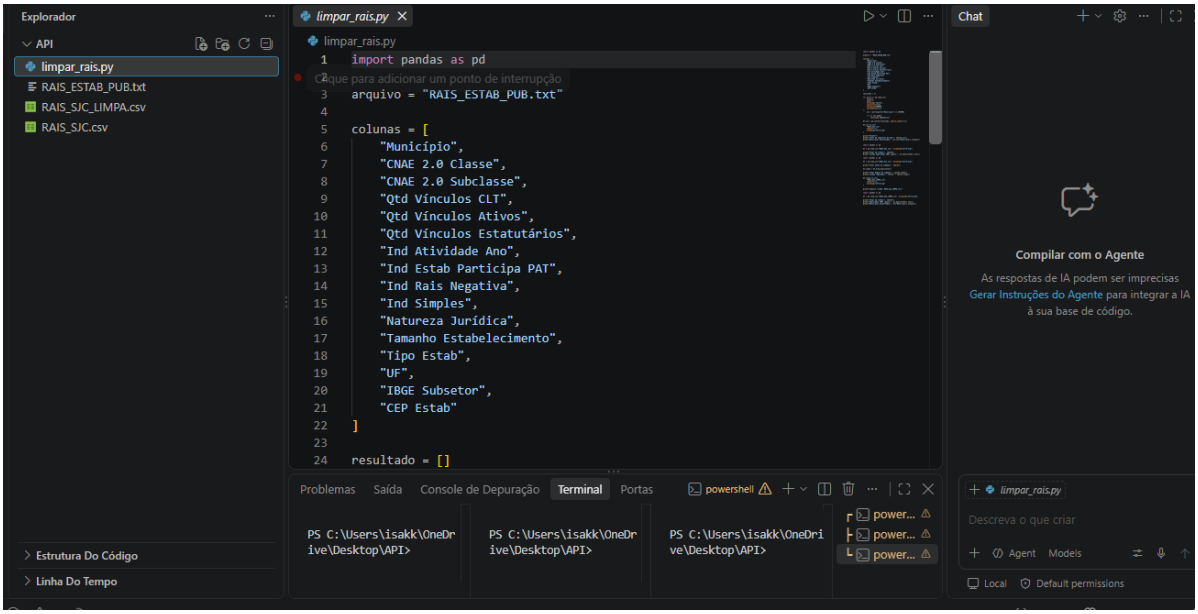


Figura 2 - Início do código, com a importação do Pandas e a lista das colunas utilizadas.

5. Leitura da base em partes

```
resultado = []

for parte in pd.read_csv(
    arquivo,
    sep=";",
    encoding="latin1",
    usecols=colunas,
    chunksize=100000,
    low_memory=False
):
```

Primeiro é criada a lista resultado. Ela começa vazia e vai recebendo as partes da base que possuem registros de São José dos Campos.

A base RAIS é muito grande, então ela não foi lida inteira de uma vez. O chunksize=100000 faz o Pandas ler 100 mil linhas por vez. Isso deixa o processo mais seguro e evita tentar carregar toda a base na memória ao mesmo tempo.

Parte do código	O que faz
sep=";"	informa que as colunas da base original são separadas por ponto e vírgula.
encoding="latin1"	é a codificação usada para o Python conseguir ler corretamente esse arquivo.
usecols=colunas	faz a leitura somente das colunas escolhidas anteriormente.
chunksize=100000	lê a base em blocos de 100 mil linhas.
low_memory=False	faz o Pandas analisar os dados de uma forma mais completa e ajuda a diminuir problemas durante a leitura das colunas.

6. Filtrando somente São José dos Campos

```
sjc = parte[parte["Município"] == 354990]

if not sjc.empty:
    resultado.append(sjc)
```

Essa é uma das partes mais importantes do código. O valor 354990 é o código usado na base RAIS para o município de São José dos Campos. Então o código olha cada parte da base e mantém somente as linhas em que a coluna Município é igual a 354990.

O if not sjc.empty verifica se naquela parte realmente foi encontrado algum registro de São José dos Campos. Quando encontra, resultado.append(sjc) coloca esses dados dentro da lista resultado.

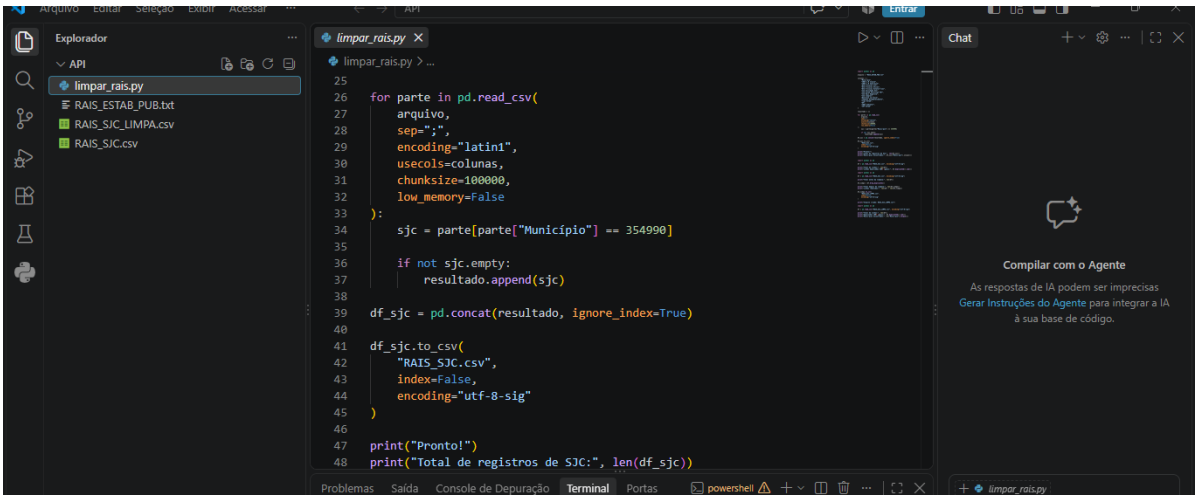


Figura 3 - Leitura da base em partes e filtro do município 354990.

7. Juntando os resultados e criando o primeiro arquivo

```
df_sjc = pd.concat(resultado, ignore_index=True)

df_sjc.to_csv(
    "RAIS_SJC.csv",
    index=False,
    encoding="utf-8-sig"
)
```

Depois que o Python termina de passar por toda a base, pd.concat junta as partes encontradas em uma única tabela. O ignore_index=True reorganiza a numeração das linhas.

Em seguida o arquivo RAIS_SJC.csv é criado. Esse é o primeiro arquivo de dados gerado pelo código e já possui somente os registros de São José dos Campos.

O index=False evita criar uma coluna extra com a numeração do Pandas. O encoding="utf-8-sig" facilita a abertura do CSV em programas como o Excel sem perder os caracteres acentuados.

```
print("Pronto!")
print("Total de registros de SJC:", len(df_sjc))
print("Municípios encontrados:", df_sjc["Município"].unique())
```

Esses prints servem para conferir o resultado no terminal. A primeira execução correta retornou 47.339 registros e somente o município 354990, confirmando que o filtro funcionou.

8. Conferência de linhas duplicadas

```
df = pd.read_csv("RAIS_SJC.csv", encoding="utf-8-sig")

print("Total de linhas:", len(df))
print("Linhas duplicadas 100% iguais:", df.duplicated().sum())
```

Depois do filtro foi feita uma conferência para saber se existiam linhas completamente repetidas. O df.duplicated() identifica linhas 100% iguais e o .sum() conta quantas foram encontradas.

O resultado foi 4.977 linhas duplicadas dentro das 47.339 linhas da base filtrada.

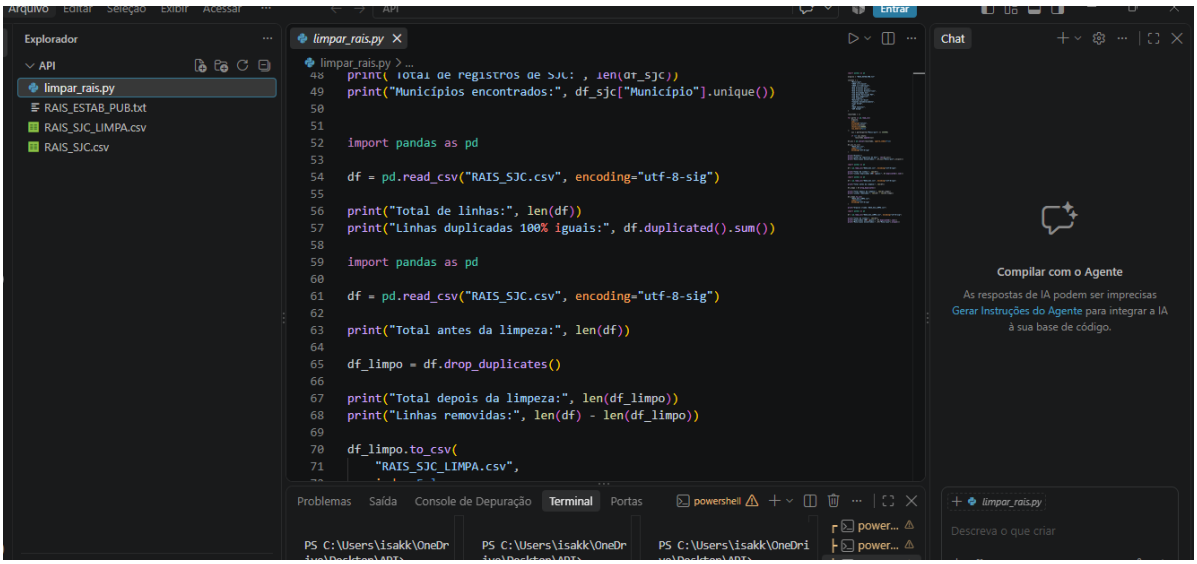


Figura 4 - Parte do código que confere os duplicados e inicia a limpeza.

9. Remoção dos duplicados

```
print("Total antes da limpeza:", len(df))

df_limpo = df.drop_duplicates()

print("Total depois da limpeza:", len(df_limpo))
print("Linhas removidas:", len(df) - len(df_limpo))
```

O drop_duplicates() remove as cópias das linhas que são completamente iguais, mantendo apenas uma delas. Antes da limpeza existiam 47.339 linhas. Depois da limpeza a base ficou com 42.362 linhas, então foram removidas 4.977 linhas 100% duplicadas.

10. Criação da base final

```
df_limpo.to_csv(
    "RAIS_SJC_LIMPA.csv",
    index=False,
    encoding="utf-8-sig"
)

print("Arquivo criado: RAIS_SJC_LIMPA.csv")
```

Essa parte cria o segundo arquivo de dados: RAIS_SJC_LIMPA.csv. Esse é o arquivo final dessa etapa, porque já contém somente São José dos Campos e não possui mais as linhas que estavam 100% duplicadas.



Figura 5 - Final do código, criação da base limpa e conferência final.

11. Conferência final

```
df = pd.read_csv("RAIS_SJC_LIMPA.csv", encoding="utf-8-sig")

print("Total de linhas:", len(df))
print("Duplicados 100% iguais:", df.duplicated().sum())
print("Municípios encontrados:", df["Município"].unique())
```

Por último, a base limpa é aberta novamente para conferir se o processo realmente funcionou. A conferência esperada é:

```
Total de linhas: 42362
Duplicados 100% iguais: 0
Municípios encontrados: [354990]
```

Com isso é possível confirmar que o arquivo final ficou somente com os registros de São José dos Campos e sem linhas 100% duplicadas.

12. Como executar pelo terminal

Com o arquivo limpar_rais.py salvo, o código precisa ser executado pelo terminal. No começo foi usado o comando:

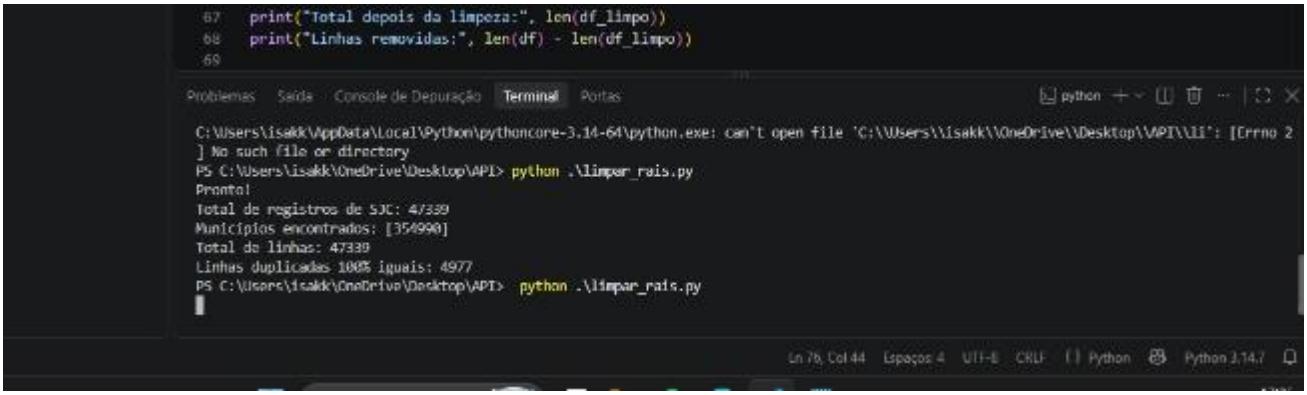
```
python limpar_rais.py
```

Mas apareceu o erro de que o arquivo não podia ser encontrado. Isso aconteceu porque o terminal estava em C:\Users\isakk e o arquivo estava dentro da pasta API.




```
cd C:\Users\isakk\OneDrive\Desktop\API
```

O comando cd serve para mudar de pasta. Depois dele o terminal passou a estar dentro da pasta API, onde estavam a base e o arquivo Python.




```
python .\limpar_rais.py
```

Esse foi o comando utilizado para executar o arquivo. O .\ indica que o arquivo está na pasta atual.

Se aparecer >> no PowerShell: significa que o terminal entendeu que o comando ainda não

terminou e está esperando continuação. Nesse caso pode ser necessário cancelar o comando e digitar novamente na linha normal do PowerShell.

Se aparecer "can't open file": confira se o terminal está na pasta correta e se o nome digitado é exatamente limpar_rais.py. (falo porque me ocorreu)

13. Arquivos gerados

Ao terminar o processo, a pasta ficou com quatro arquivos principais: a base original, o código Python e os dois CSVs gerados durante a limpeza.

- RAIS_ESTAB_PUB.txt - base original.
- limpar_rais.py - código utilizado no processo.
- RAIS_SJC.csv - base somente com São José dos Campos, antes da retirada dos duplicados.
- RAIS_SJC_LIMPA.csv - base final de São José dos Campos, sem linhas 100% duplicadas.

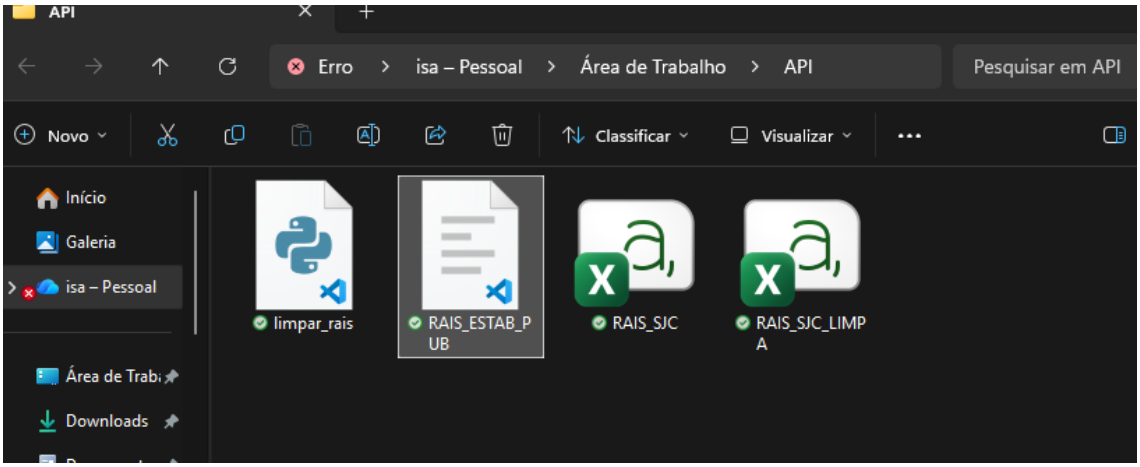


Figura 6 - Arquivos existentes na pasta depois da execução do código.

14. Visualização dos arquivos no Excel

Os arquivos CSV podem ser abertos no Excel. Na abertura direta, dependendo da configuração do Excel, os dados podem aparecer todos na mesma coluna separados por vírgulas. Isso não quer dizer que a limpeza deu errado; é apenas a forma como o Excel interpretou o delimitador do arquivo CSV.

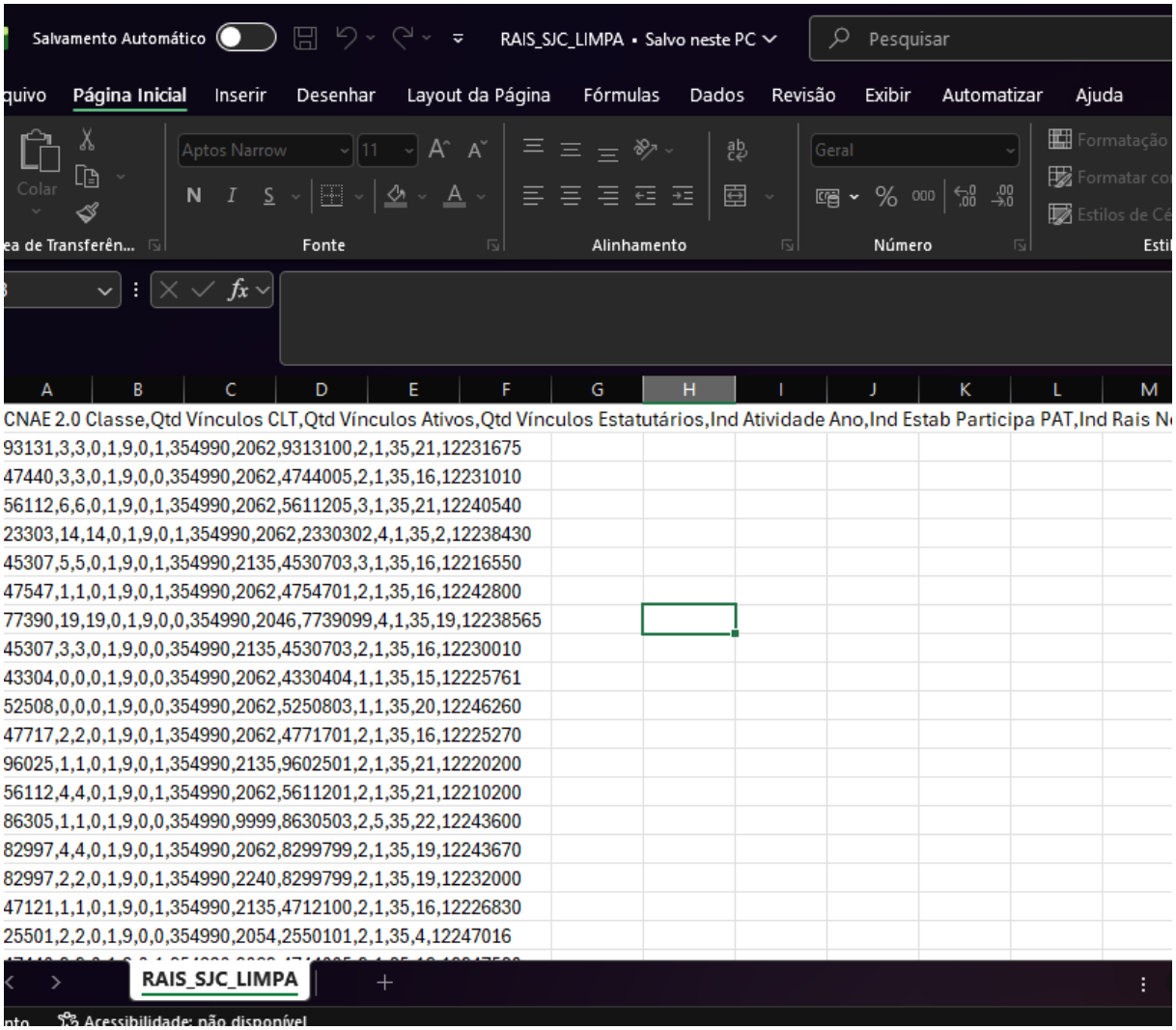


Figura 7 - Arquivo RAIS_SJC_LIMPA.csv aberto diretamente no Excel.

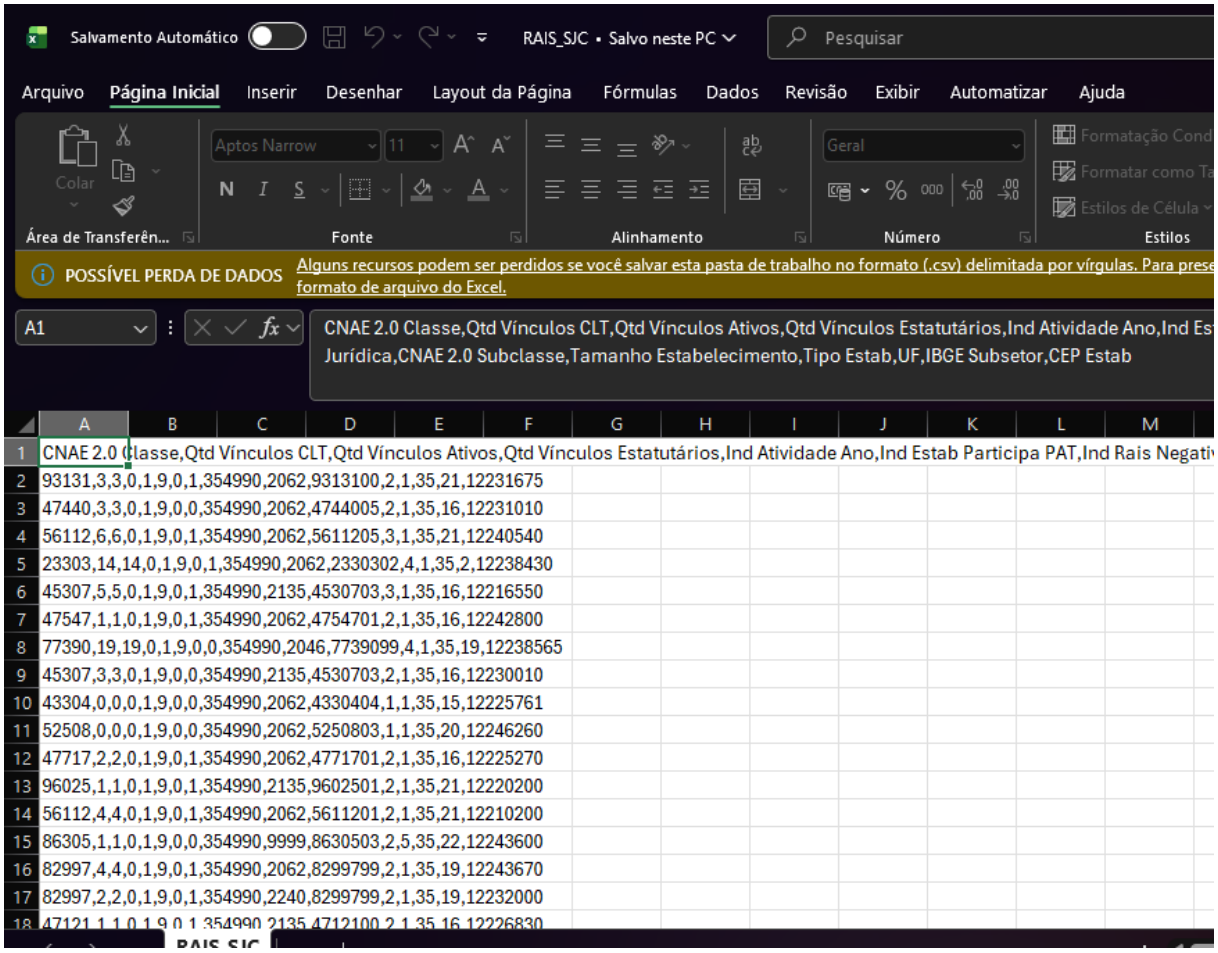


Figura 8 - Arquivo RAIS_SJC.csv aberto diretamente no Excel.

Observação: para utilizar a base no Power BI ou organizar as colunas no Excel, o arquivo pode ser importado escolhendo a vírgula como delimitador. A etapa deste manual termina na criação da base limpa; a organização e classificação dos códigos fica para a etapa seguinte do projeto.

15. Resultado final

Conferência	Resultado
Registros filtrados de SJC	**47.339**
Município encontrado	**354990**
Duplicados 100% iguais encontrados	**4.977**
Duplicados removidos	**4.977**
Registros finais	**42.362**
Arquivo final	**RAIS_SJC_LIMPA.csv**

No final dessa etapa foi obtida uma base limpa com 42.362 registros somente de São José dos Campos. A partir dessa base seria realizada a próxima parte do projeto, que consiste em identificar e

traduzir os códigos CNAE, natureza jurídica, subsetor do IBGE e outras informações necessárias para montar os gráficos, mapas e análises no Power BI.